

Machine Learning for Business

GTSC2143

Lecture 10 Clustering II

Instructor: Dr. Lily Bei Luo

AY2025-2026 Semester 1

Review: Clustering documents

Clustering documents

Structure documents by topic

- Discover groups (*clusters*) of related articles



SPORTS

WORLD NEWS

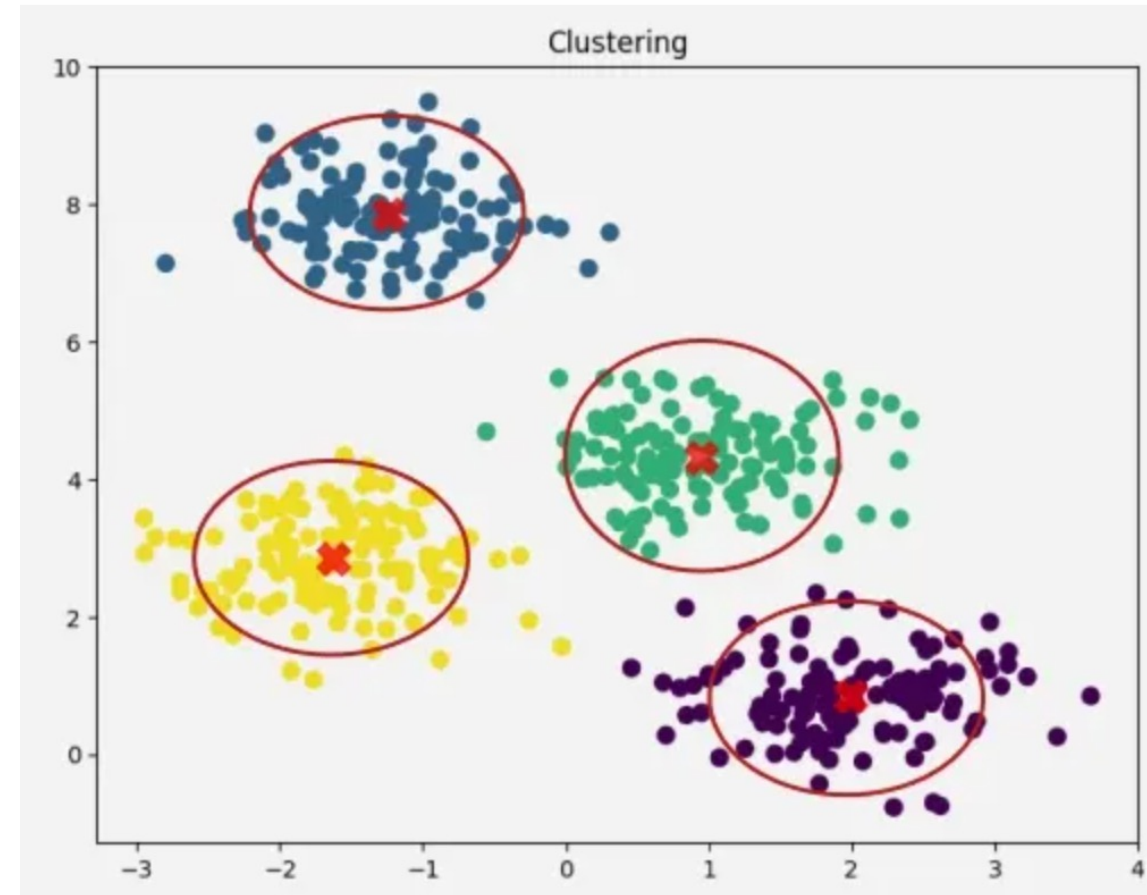
Clustering Overview & Classic Algorithms

Clustering in Machine Learning

What is clustering?

Clustering is an **unsupervised** machine learning technique that groups similar data points together into clusters based on their characteristics, **without using any labeled data**.

- **Goal:** Discover the natural grouping or structure in unlabeled data without predefined categories.
- **How:** Data points are assigned to clusters based on **similarity or distance metrics**.
- **Output:** Each group is assigned a cluster ID, representing shared characteristics within the cluster.

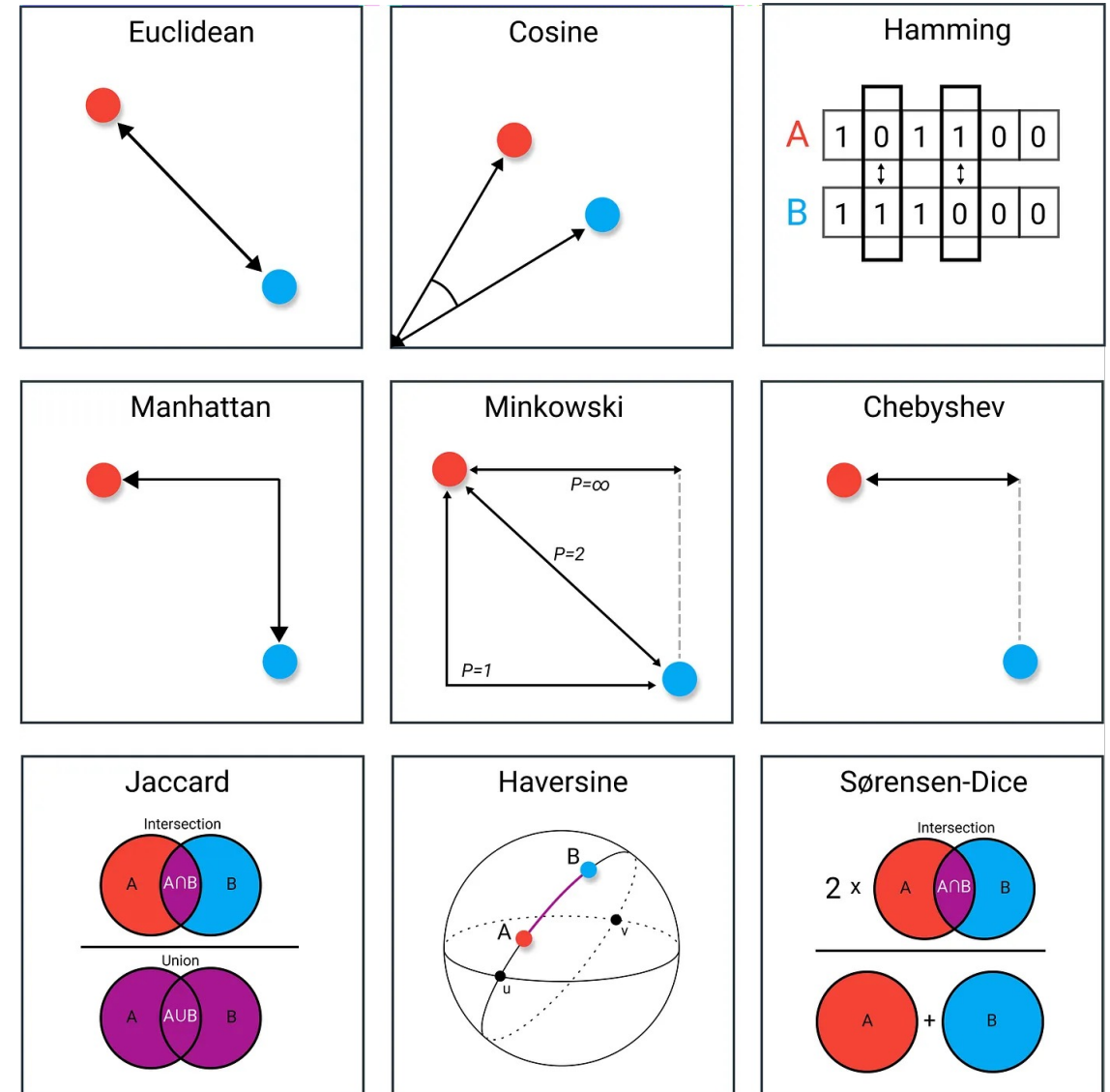


Measuring Similarity

Distance metrics

Distance metrics are simple mathematical formulas to measure **how similar or different two data points are**.

- Type of distance metrics we choose plays a big role in deciding clustering results.
- *Knowing which distance metric to apply in different situations can transform a messy clustering outcome into clear and interpretable clusters!*



Measuring Similarity

Euclidean distance

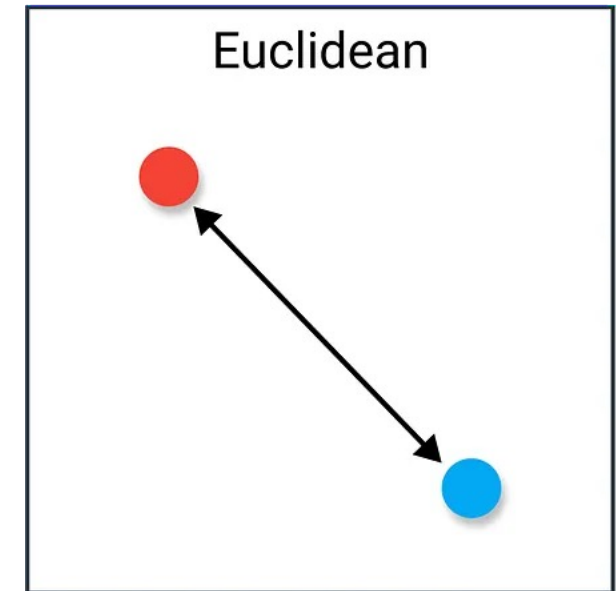
Euclidean distance is defined as the distance between two points in Euclidean space. It is the most widely used distance metric.

Consider two points $A(x_1, y_1)$ and $B(x_2, y_2)$ in a 2-dimensional space; the Euclidean distance between them is given by using the formula:

$$d(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

where,

- (x_1, y_1) is the Coordinate of the first point,
- (x_2, y_2) is the Coordinate of the second point.



Measuring Similarity

Manhattan distance

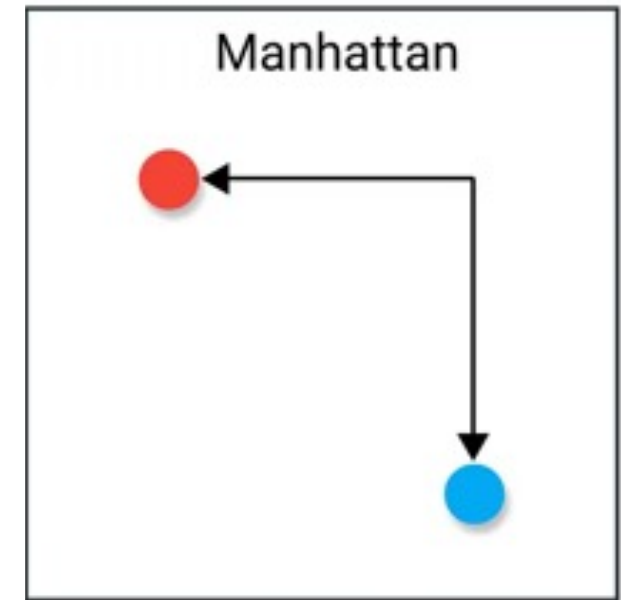
Manhattan distance measures the distance between two points by calculating the sum of the absolute differences of their corresponding coordinates.

Consider two points $A(x_1, y_1)$ and $B(x_2, y_2)$ in a 2-dimensional space; the Manhattan distance between them is given by using the formula:

$$d(A, B) = |x_1 - x_2| + |y_1 - y_2|$$

where,

- (x_1, y_1) is the Coordinate of the first point,
- (x_2, y_2) is the Coordinate of the second point.



Measuring Similarity

Chebyshev distance

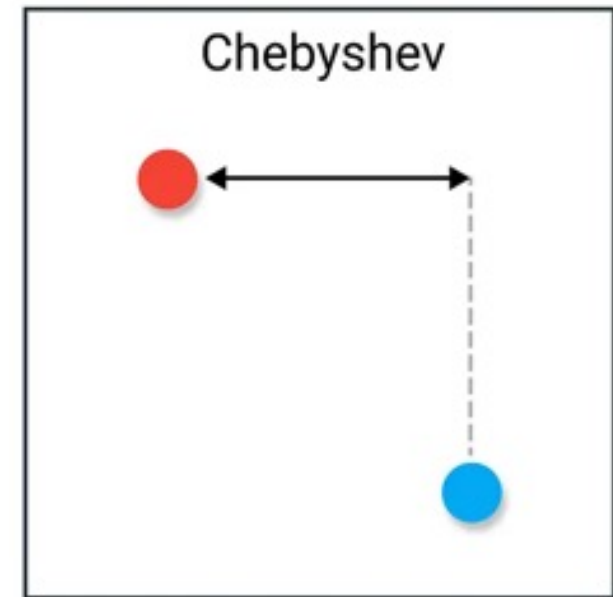
Chebyshev distance measures the greatest difference along any coordinate dimension between two points.

Consider two points $A(x_1, y_1)$ and $B(x_2, y_2)$ in a 2-dimensional space; the Chebyshev distance between them is given by using the formula:

$$d(A, B) = \max(|x_1 - x_2|, |y_1 - y_2|)$$

where,

- (x_1, y_1) is the Coordinate of the first point,
- (x_2, y_2) is the Coordinate of the second point.



Measuring Similarity

Chebyshev distance

Chebyshev distance is often called “chessboard distance” or “king’s move distance”. It essentially asks:

➡ If you can move along all dimensions simultaneously, what is the minimum number of steps needed to reach the target point?

- On a chessboard, the **king** can move one square in any direction (horizontally, vertically, or diagonally).
- The number of moves required for the king to travel from point A to point B equals the **maximum** of the horizontal and vertical differences.

	a	b	c	d	e	f	g	h	
8	5	4	3	2	2	2	2	2	8
7	5	4	3	2	1	1	1	2	7
6	5	4	3	2	1		1	2	6
5	5	4	3	2	1	1	1	2	5
4	5	4	3	2	2	2	2	2	4
3	5	4	3	3	3	3	3	3	3
2	5	4	4	4	4	4	4	4	2
1	5	5	5	5	5	5	5	5	1
	a	b	c	d	e	f	g	h	

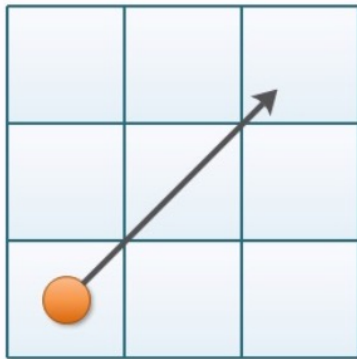
Question: Why the Chebyshev distance between f6 and d2 is 4?

Measuring Similarity

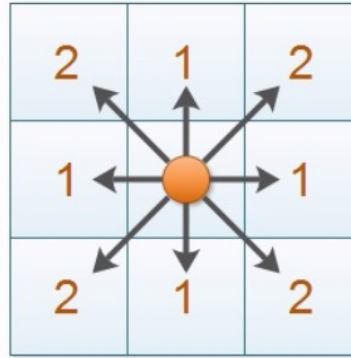
Visual Difference

This image summarizes the difference in Euclidean, Manhattan, and Chebyshev distances.

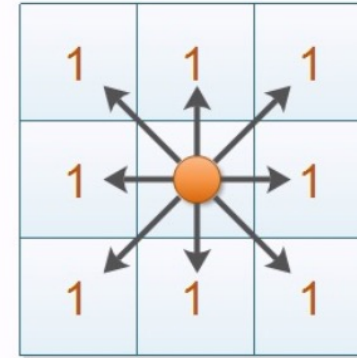
Euclidean Distance



Manhattan Distance



Chebyshev Distance



$$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2} \quad |x_1 - x_2| + |y_1 - y_2| \quad \max(|x_1 - x_2|, |y_1 - y_2|)$$

Measuring Similarity

Minkowski distance

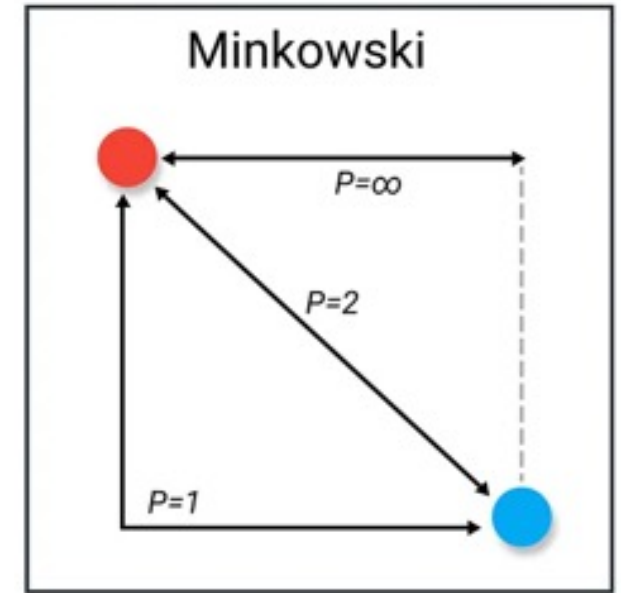
Minkowski distance is a generalized distance metric that includes the Euclidean, Manhattan, and Chebyshev distances.

Consider two points $A(x_1, y_1)$ and $B(x_2, y_2)$ in a 2-dimensional space; the Minkowski distance between them is given by using the formula:

$$d(A, B) = (|x_1 - x_2|^p + |y_1 - y_2|^p)^{\frac{1}{p}}$$

where,

- p is the order of the Minkowski distance,
- (x_1, y_1) is the Coordinate of the first point,
- (x_2, y_2) is the Coordinate of the second point.



Note:

- When $p = 1$, it becomes the Manhattan distance.
- When $p = 2$, it becomes the Euclidean distance.
- When $p \rightarrow \infty$, it becomes the Chebyshev distance.

Measuring Similarity

Cosine similarity

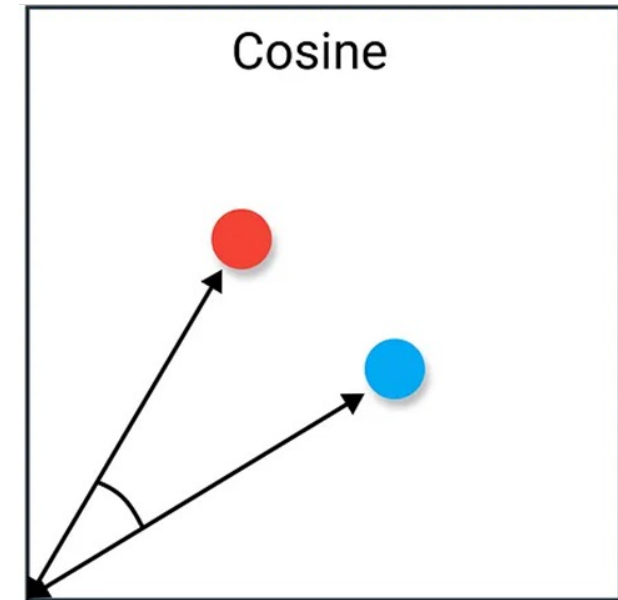
Cosine similarity measures the similarity between two non-zero vectors by calculating the cosine of the angle between them.

Consider two points $A(x_1, y_1)$ and $B(x_2, y_2)$ in a 2-dimensional space; the cosine similarity between them is given by using the formula:

$$d(A, B) = \frac{x_1x_2 + y_1y_2}{\sqrt{x_1^2 + y_1^2}\sqrt{x_2^2 + y_2^2}}$$

where,

- (x_1, y_1) is the Coordinate of the first point,
- (x_2, y_2) is the Coordinate of the second point.



Measuring Similarity

Jaccard distance

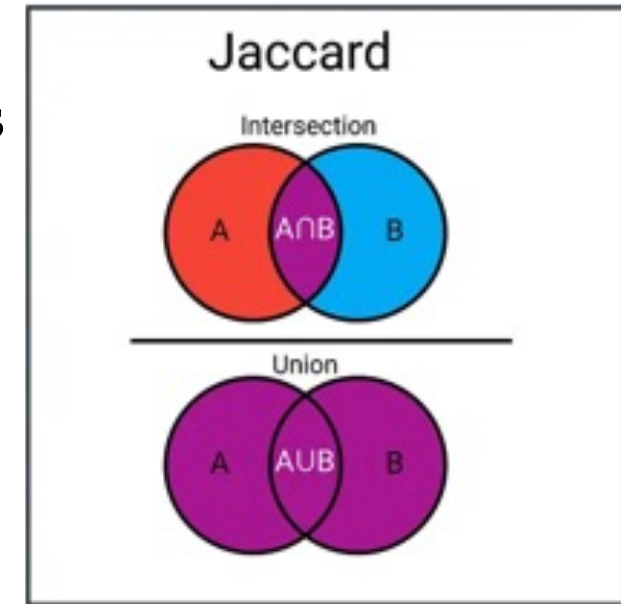
Jaccard distance measures the dissimilarity between two sets by comparing the size of their intersection with the size of their union.

Consider two sets **A** and **B**; the Jaccard distance between them is given by using the formula:

$$d(A, B) = 1 - \frac{|A \cap B|}{|A \cup B|}$$

where,

- $|A \cap B|$ is the number of elements common to both sets,
- $|A \cup B|$ is the number of elements present in either of the two sets.



Measuring Similarity

Sørensen–Dice distance

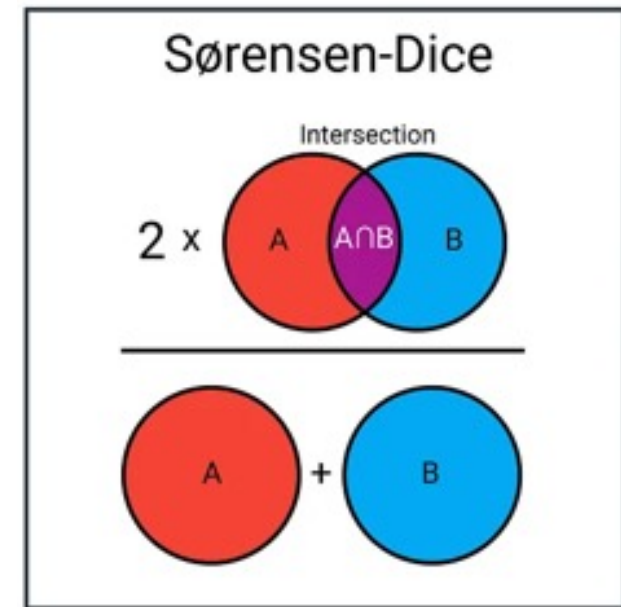
Sørensen–Dice distance measures the dissimilarity between two sets based on the size of their intersection relative to their total size.

Consider two sets **A** and **B**; the Sørensen–Dice distance between them is given by using the formula:

$$d(A, B) = 1 - 2 \frac{|A \cap B|}{|A| + |B|}$$

where,

- $|A|$ is the number of elements in set A,
- $|B|$ is the number of elements in set B,
- $|A \cap B|$ is the number of elements common to both sets.



Measuring Similarity

Haversine distance

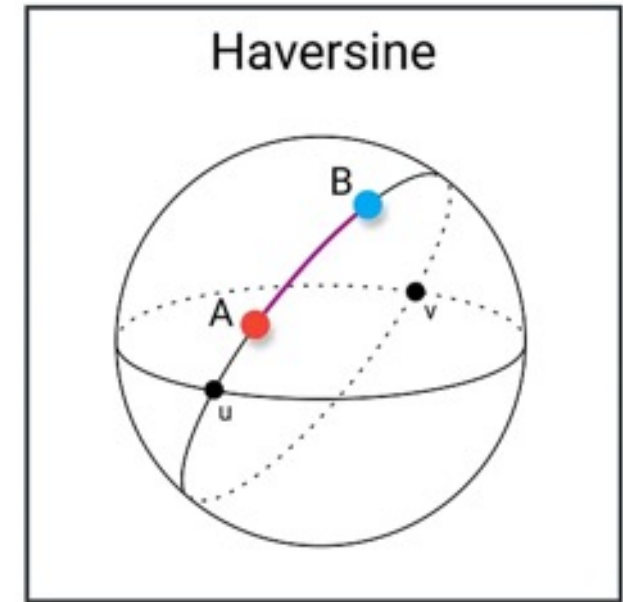
Haversine distance measures the shortest great-circle distance between two points **on the surface of a sphere**, given their latitudes and longitudes.

Consider two points **A**(lat₁, lon₁) and **B**(lat₂, lon₂) on Earth's surface; the Haversine distance between them is given by using the formula:

$$d(A, B) = 2r \arcsin\left(\sqrt{\sin^2\left(\frac{\text{lat}_2 - \text{lat}_1}{2}\right) + \cos(\text{lat}_1)\cos(\text{lat}_2)\sin^2\left(\frac{\text{lon}_2 - \text{lon}_1}{2}\right)}\right)$$

where,

- r is the radius of the Earth (mean radius $\approx 6,371$ km),
- (lat₁, lon₁) are the coordinates (latitude and longitude) of the first point,
- (lat₂, lon₂) are the coordinates of the second point.



Measuring Similarity

Hamming distance

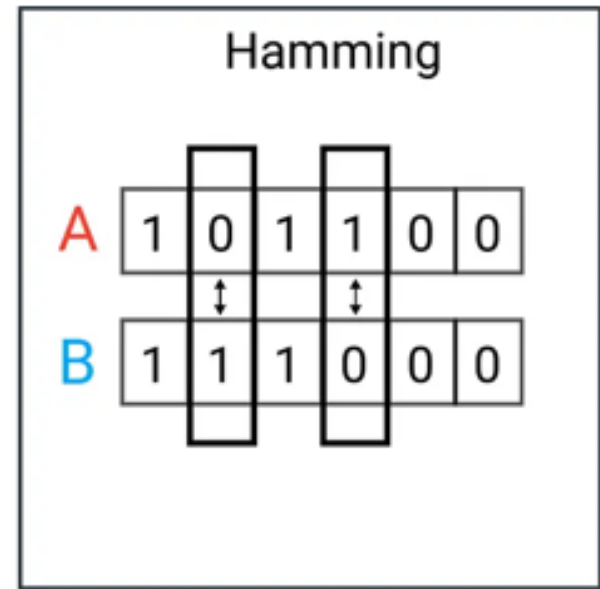
Hamming distance measures the number of positions at which the corresponding symbols of two equal-length strings are different. It is commonly used to compare binary strings or categorical data.

Consider two strings **A** and **B** of equal length n ; the Hamming distance between them is given by using the formula:

$$d(A, B) = \sum_{i=1}^k |A_i - B_i|$$

where,

- A_i and B_i are the symbols (characters or bits) at position i in strings A and B,
- $|A_i \neq B_i|$ equals 1 if the symbols differ, and 0 if they are the same.



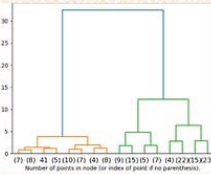
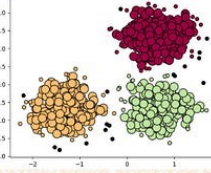
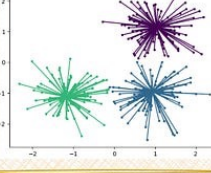
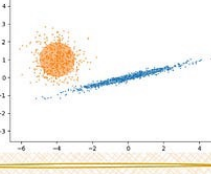
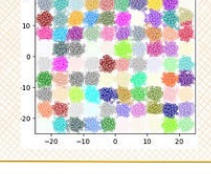
From Distances to Clusters

Clustering algorithms

Clustering algorithms are unsupervised learning methods used to **group similar data points into cluster**.

- Clustering methods can be classified on the basis of how they form clusters.



Type of Clustering Algorithm	Visual Overview	Description	Algorithm(s)
Centroid-based		Cluster points based on proximity to centroid	KMeans KMeans++ KMedoids
Connectivity-based		Cluster points based on proximity between clusters	Hierarchical Clustering (Agglomerative and Divisive)
Density-based		Cluster points based on their density instead of proximity	DBSCAN OPTICS HDBSCAN
Graph-based		Cluster points based on graph distance	Affinity Propagation Spectral Clustering
Distribution-based		Cluster points based on their likelihood of belonging to the same distribution.	Gaussian Mixture Models
Compression-based		Transform data to a lower dimensional space and then perform clustering	BIRCH

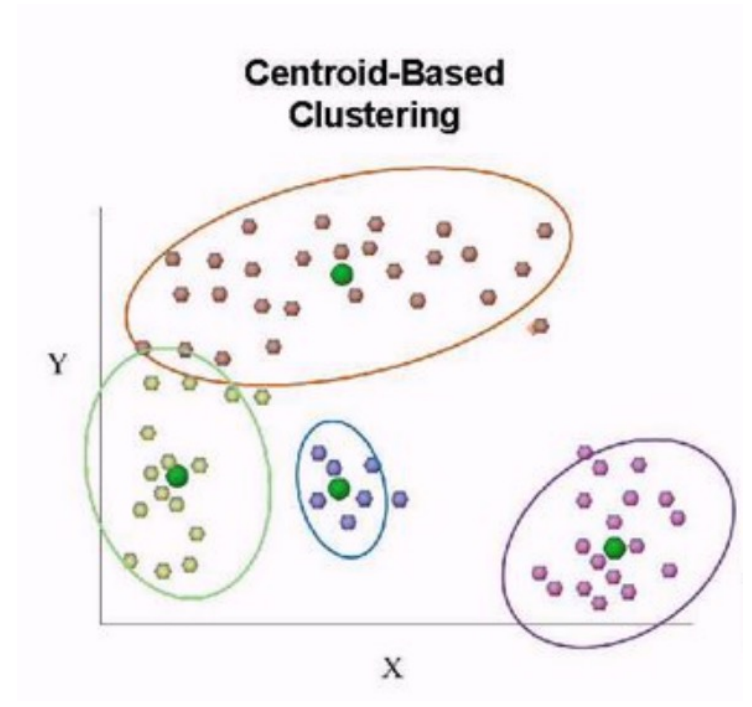
From Distances to Clusters

Centroid-based clustering (Partitioning methods)

Centroid-based clustering organizes data points around central prototypes called **centroids**, where each cluster is represented by the mean (or medoid) of its members.

The number of clusters is specified in advance and the algorithm allocates points to the nearest centroid.

- **K-Means**: Iteratively assigns points to nearest centroid and recalculates centroids to minimize intra-cluster variance.
- **K-Medoids**: Similar to K-Means but uses actual data points (medoids) as centers, robust to outliers.



From Distances to Clusters

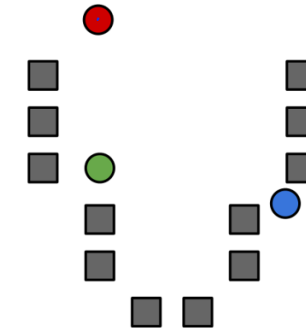
K-Means clustering

K-Means is one of the most widely used unsupervised machine learning algorithms for partitioning a dataset into **k distinct, non-overlapping** clusters.

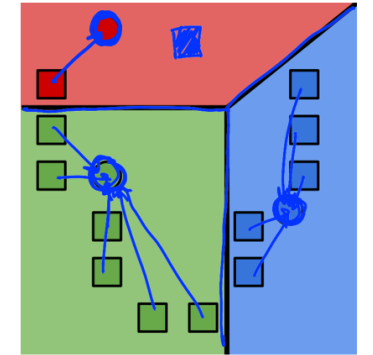
The basic steps of the K-Means algorithm are as follows:

1. **Initialization:** Choose the number of clusters (k) and randomly select k initial centroids.
2. **Assignment:** Assign each data point to the nearest centroid based on a distance measure (**commonly Euclidean distance**).
3. **Update:** Recalculate each centroid as the mean of all points assigned to that cluster.
4. **Repeat:** Continue the assignment and update steps until the centroids stabilize (no significant change).

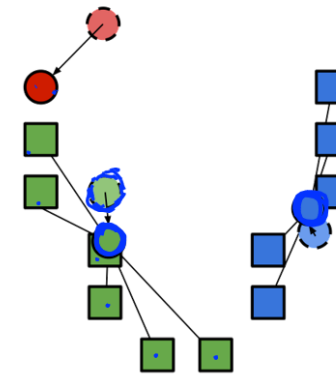
Remember? We learned last class!



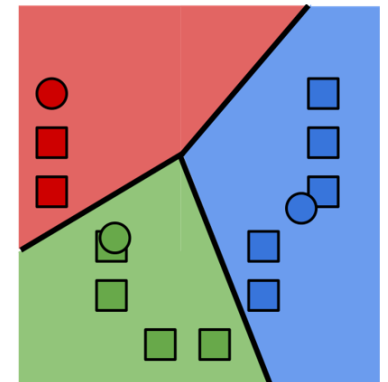
1. Initialization



2. Assignment



3. Update



4. Repeat

From Distances to Clusters

K-Means clustering

K-Means is one of the most widely used unsupervised machine learning algorithms for partitioning a dataset into **k distinct, non-overlapping** clusters.

Pros:

- Fast and scalable for large datasets.
- Simple to implement and interpret.

Cons:

- **Requires pre-knowledge of k.**
- Sensitive to initialization and outliers.
- Not suitable for non-spherical clusters.

We use **Elbow Method** for optimal value of k in K-Means, which is a graphical tool used to determine the optimal number of clusters (k) in K-Means.

From Distances to Clusters

Elbow Method for optimal value of k in K-Means

The Elbow Method helps by plotting the Within-Cluster Sum of Squares (WCSS) against increasing k values and looking for a point where the improvement slows down, this point is called the "elbow."

$$\text{WCSS} = \sum_{k=1}^K \sum_{i \in C_k} \|x_i - \mu_k\|^2$$

where,

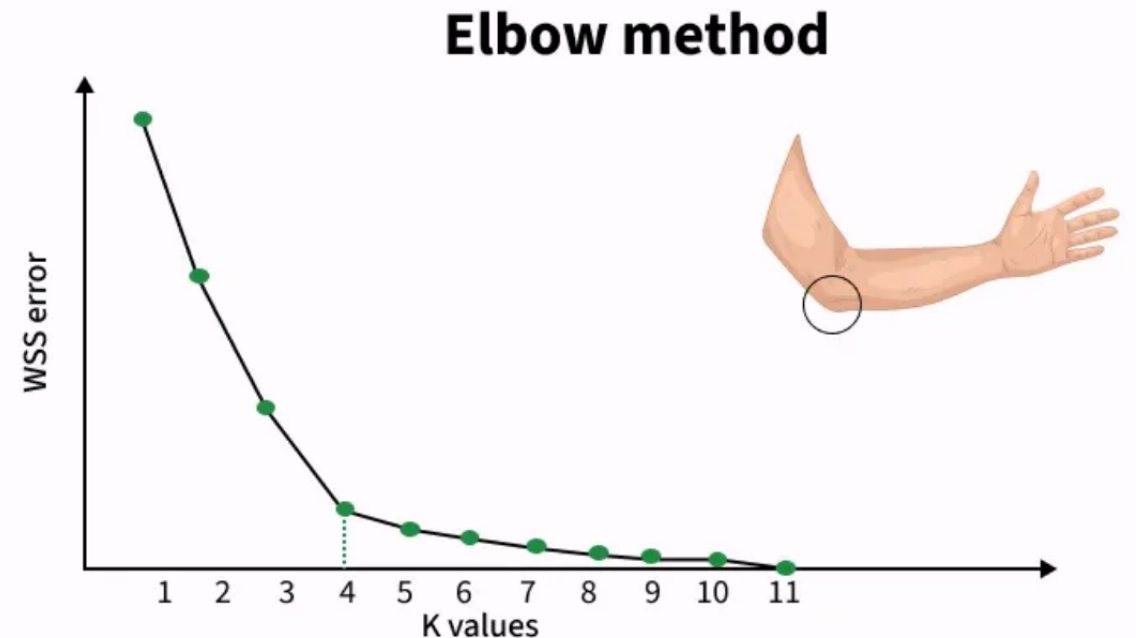
K : the number of clusters,

C_k : the set of data points assigned to cluster k ,

x_i : a data point in cluster k ,

μ_k : the centroid of cluster k ,

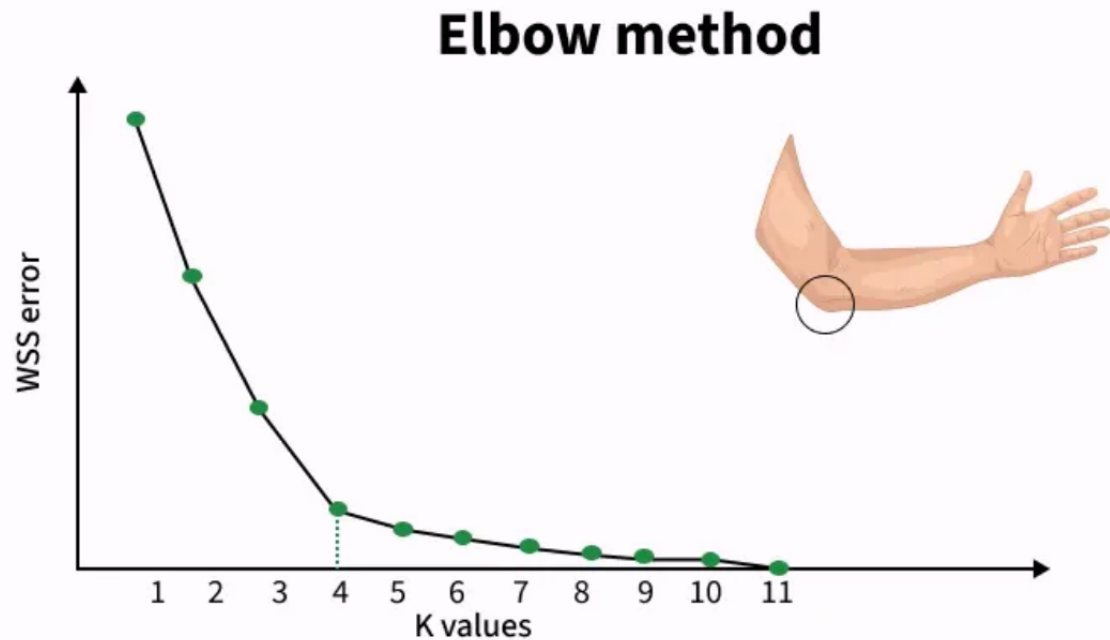
$\|x_i - \mu_k\|^2$: the squared Euclidean distance between the point and its cluster centroid.



From Distances to Clusters

Elbow Method for optimal value of k in K-Means

The Elbow Method helps by plotting the Within-Cluster Sum of Squares (WCSS) against increasing k values and looking for a point where the improvement slows down, this point is called the "elbow."



- **Before the elbow:** WCSS drops quickly -> clusters become much better.
- **After the elbow:** WCSS drops slowly -> extra clusters add little value and may lead to overfitting.

From Distances to Clusters

K-Means++ clustering

Note:

- K-Means++ reduces the likelihood of poor clustering results caused by random initialization.
- It achieves better cluster quality and faster convergence than traditional K-Means.

K-Means++ is an improved version of the traditional K-Means algorithm that focuses on **better initialization of cluster centroids**.

- The goal of K-Means++ is to choose initial cluster centers that are **more spread out**, helping the algorithm converge faster and produce more stable, accurate results.

The basic steps of the K-Means++ algorithm are as follows:

1. **Initialization (First Centroid):** Randomly choose one data point from the dataset as the first centroid.
2. **Select Remaining Centroids:** For each remaining data point, compute its distance $d(x)$ from the nearest centroid already chosen. Then, select the next centroid with probability proportional to $d(x)^2$, which means **points farther from existing centroids are more likely to be chosen**.
3. **Repeat:** Continue selecting centroids until k initial centroids have been chosen.
4. **Run K-Means:** Use these centroids as the starting points for the standard K-Means algorithm

From Distances to Clusters

K-Medoids clustering

Note:

- K-Medoids is more robust to noise and outliers compared to K-Means, since it uses actual data points as cluster centers rather than arithmetic means.

K-Medoids is similar to K-Means, but **instead of using the mean (centroid)** as the cluster center, K-Medoids **uses an actual data point (medoid)** that minimizes the total distance to other points in the same cluster.

The basic steps of the K-Medoids algorithm are as follows:

1. Initialization: Choose the number of clusters (k) and randomly select k data points as the initial **medoids**.
2. Assignment: Assign each data point to the **nearest medoid** based on a chosen distance measure (e.g., Manhattan or Euclidean distance).
3. Update: For each cluster, **select a new medoid** that minimizes the sum of distances between the medoid and all points in the cluster.
4. Repeat: Continue the assignment and update steps **until the medoids no longer change significantly or a convergence condition is met**.

From Distances to Clusters

Comparison of common K-Means variants

Algorithm	Center Type	Key Idea / Feature	Advantages	Limitations	Typical Use Cases
K-Means	Mean (centroid)	Iteratively minimizes within-cluster variance (WCSS)	Simple, fast, widely used	Sensitive to outliers & initialization	Numerical continuous data, basic clustering
K-Means++	Mean (centroid)	Smart initialization using distance-based probability	Better convergence, fewer empty clusters	Slightly slower initialization	Most general-purpose K-Means tasks
K-Medoids (PAM)	Actual data points (medoids)	Minimizes total pairwise distance	Robust to noise and outliers	Higher computational cost	Datasets with outliers or categorical data
Hybrid K-Means–K-Medoids	Mean + Medoid combined	Uses K-Means speed + K-Medoids robustness	Good accuracy–speed balance, robust	More complex implementation	Data with moderate noise or mixed types
...	...	...	...	...	...

From Distances to Clusters



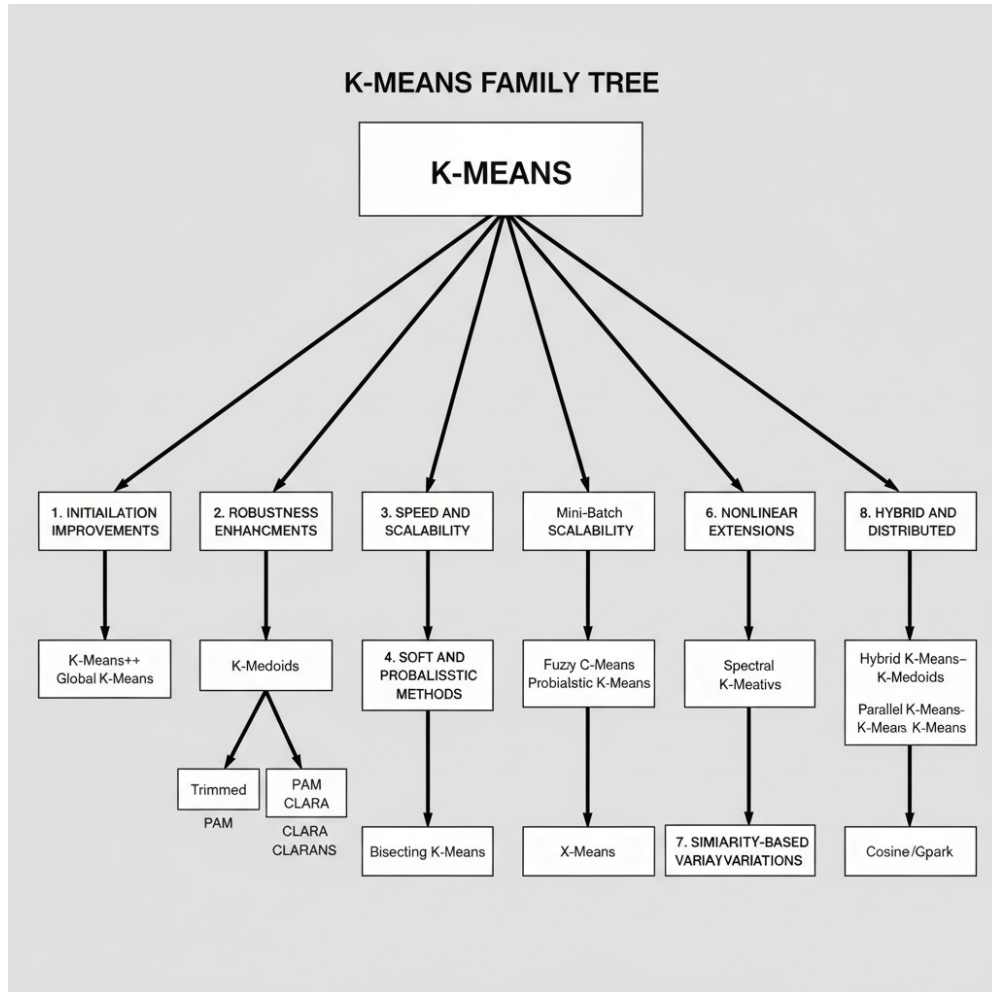
Distance metrics in common K-Means variants

Algorithm	Default Distance Metric	Can Be Replaced?	Typical Alternatives / Use Cases
K-Means	Euclidean distance (L2)	Limited	By default, K-Means assumes the Euclidean (L2) distance to minimize the sum of squared errors
K-Means++	Same as K-Means	Yes	Any metric consistent with chosen clustering objective
K-Medoids (PAM)	Any chosen metric (usually Manhattan or Euclidean)	Highly flexible	Works with any distance or dissimilarity measure (even non-Euclidean)
Hybrid K-Means–K-Medoids	Euclidean or Manhattan (mixed)	Yes	Often hybrid — mean step uses Euclidean, medoid step uses Manhattan

- Most K-Means variants allow different distance metrics, but the standard K-Means fundamentally assumes the Euclidean (L_2) distance, since the mean minimizes squared errors.
- Using another distance metric requires redefining the objective function and the way cluster centers are computed.

From Distances to Clusters

K-Means family



K-Means Evolution

- K-Means has evolved into a broad family of clustering algorithms.
- Its core principles (minimizing intra-cluster variation and using centroids or representative points) are highly adaptable.
- Researchers have proposed **over 50** major K-Means variants. Among these, about 10–15 are widely used, such as:
 - K-Means, K-Means++, Mini-Batch K-Means, K-Medoids, CLARA, Fuzzy C-Means, Kernel K-Means, Bisecting K-Means, Spherical K-Means, X-Means, etc.

From Distances to Clusters

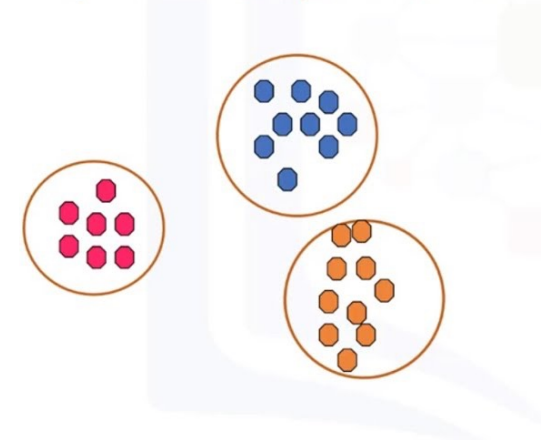
Density-based clustering (Model-based methods)

Density-based clustering defines clusters as contiguous regions of high data density separated by areas of lower density.

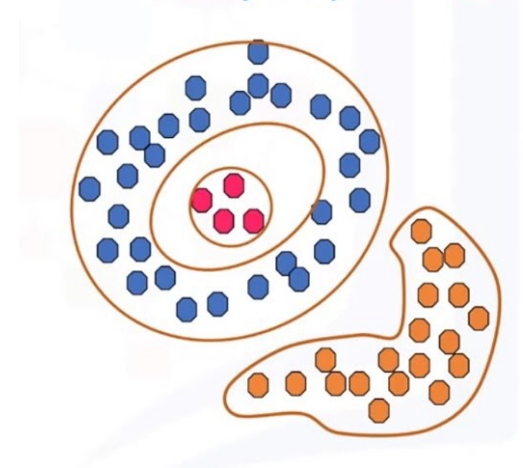
This approach can identify clusters of **arbitrary shapes**, **handles noise well** and does not require predefining the number of clusters, though its effectiveness depends on chosen density parameters.

- **DBSCAN** (Density-Based Spatial Clustering of Applications with Noise): Groups points with sufficient neighbors; labels sparse points as noise.
- **OPTICS** (Ordering Points To Identify Clustering Structure): Extends DBSCAN to handle varying densities.

• Spherical-shape clusters



• Arbitrary-shape clusters



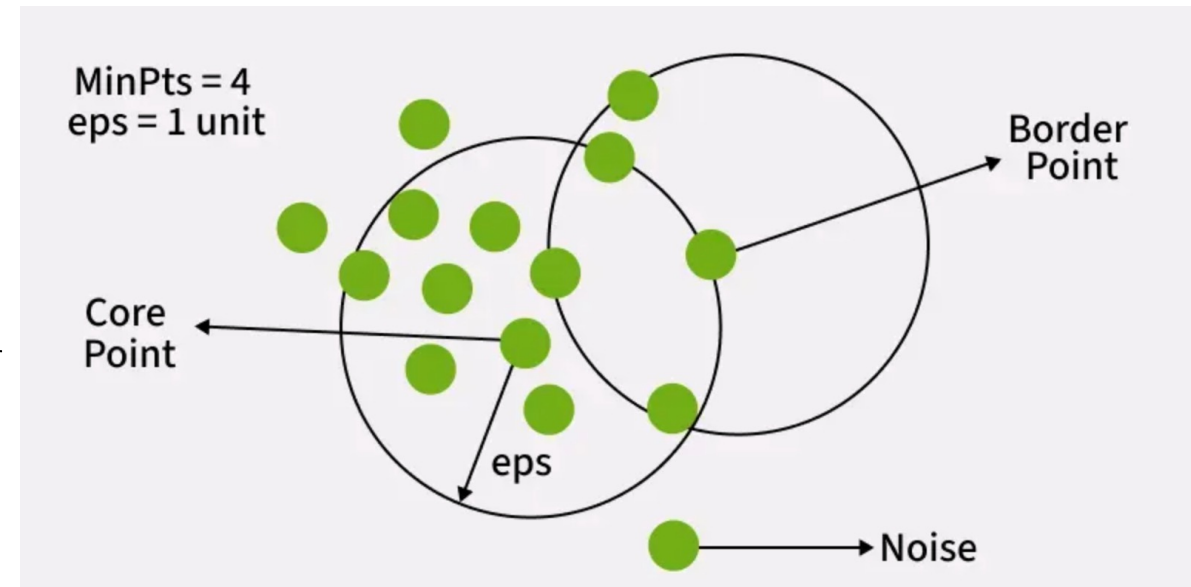
From Distances to Clusters

DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a density-based clustering algorithm that groups closely packed points into clusters and identifies isolated points as noise. It does **not require specifying the number of clusters (k)** in advance and can form clusters of **arbitrary shapes**.

Key Parameters

- **eps (ϵ): Neighborhood radius.**
 - Too small $\rightarrow$ many points labeled as noise.
 - Too large $\rightarrow$ clusters may merge.
- **MinPts: Minimum number of neighbors to form a dense region.**
 - Rule of thumb: $\text{MinPts} \geq D + 1$ (D = data dimensions).
 - Common default: $\text{MinPts} = 3$.



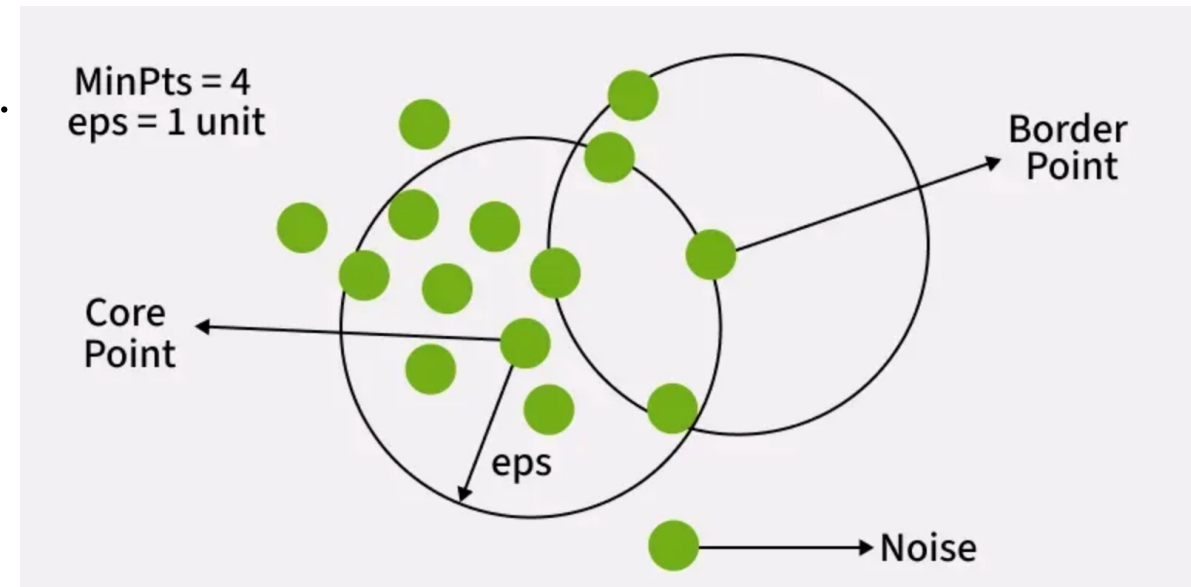
From Distances to Clusters

DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a density-based clustering algorithm that groups closely packed points into clusters and identifies isolated points as noise. It does **not require specifying the number of clusters (k)** in advance and can form clusters of **arbitrary shapes**.

How does DBSCAN work?

- **Core Points:** Have $\geq \text{MinPts}$ neighbors within eps .
- **Border Points:** Near a core point but with fewer neighbors.
- **Noise Points:** Not part of any cluster.
 - Clusters form by expanding from core points through density-reachable neighbors.



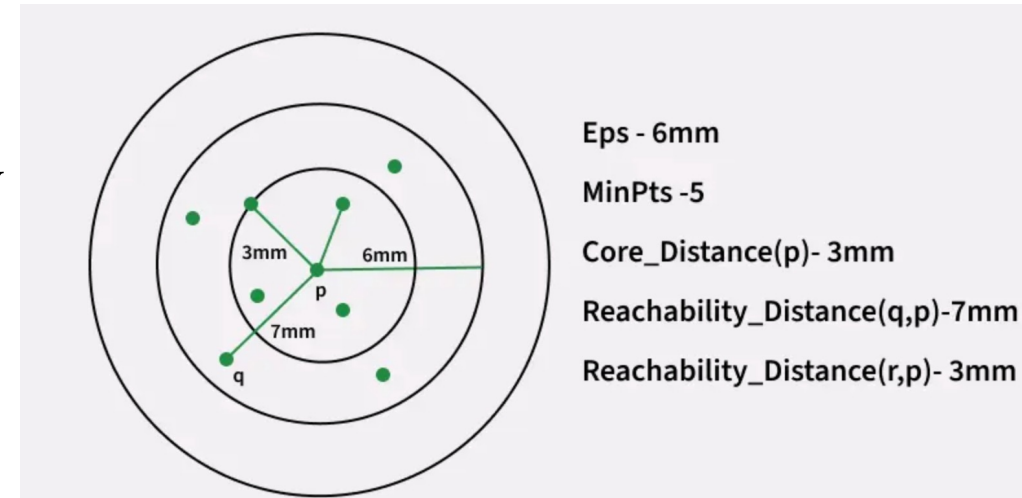
From Distances to Clusters

OPTICS

OPTICS (Ordering Points To Identify the Clustering Structure) is a density-based clustering algorithm that builds upon DBSCAN to **overcome its sensitivity to the *eps* parameter**.

Key Parameters

- **MinPts:** Minimum number of neighbors to form a dense region.
- **eps (ϵ):** Neighborhood radius (**optional**), instead of defining fixed clusters directly, OPTICS computes two key concepts for each point
 - **Core Distance:** The smallest radius ϵ that makes a point a core point (i.e., contains at least MinPts neighbors).
 - **Reachability Distance:** The distance needed to reach a point from another core point based on density connectivity.



From Distances to Clusters

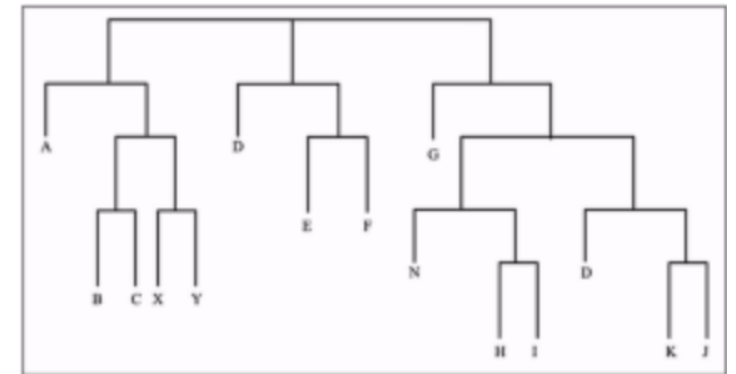
Connectivity-based clustering (Hierarchical clustering)

Connectivity-based (or hierarchical) clustering builds nested groupings of data by evaluating how data points are connected to their neighbors.

It creates a dendrogram **a tree-like structure** that reflects relationships at various granularity levels and does **not require specifying cluster numbers in advance**, but can be computationally intensive.

Approaches:

- **Agglomerative (Bottom-up):** Start with each point as a cluster; iteratively merge closest clusters.
- **Divisive (Top-down):** Start with one cluster; iteratively split into smaller clusters.



From Distances to Clusters

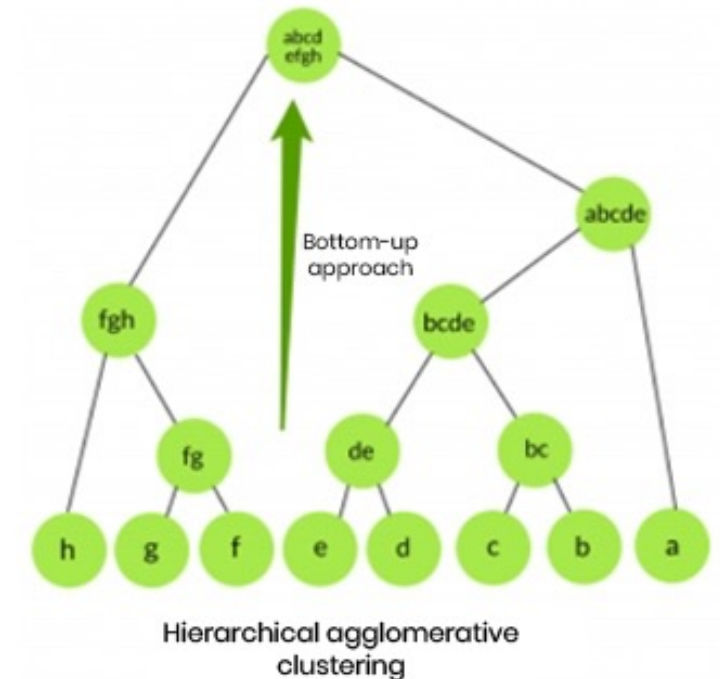
Agglomerative hierarchical clustering

Agglomerative clustering builds a cluster hierarchy from **the bottom up**.

Key Steps :

1. **Initialization:** Treat each data point as its own cluster.
2. **Compute Distances:** Calculate the pairwise distances between all clusters using a distance measure (e.g., Euclidean, Manhattan, or Cosine).
3. **Merge Clusters:** Find the two clusters with the smallest distance and merge them into one.
4. **Update Distances:** Recalculate the distances between the new cluster and all remaining clusters according to a chosen linkage criterion. [computational cost, $O(n^3)$]
5. **Repeat:** Continue merging until a stopping condition is met (e.g., a target number of clusters or a distance threshold).

Agglomerative Clustering



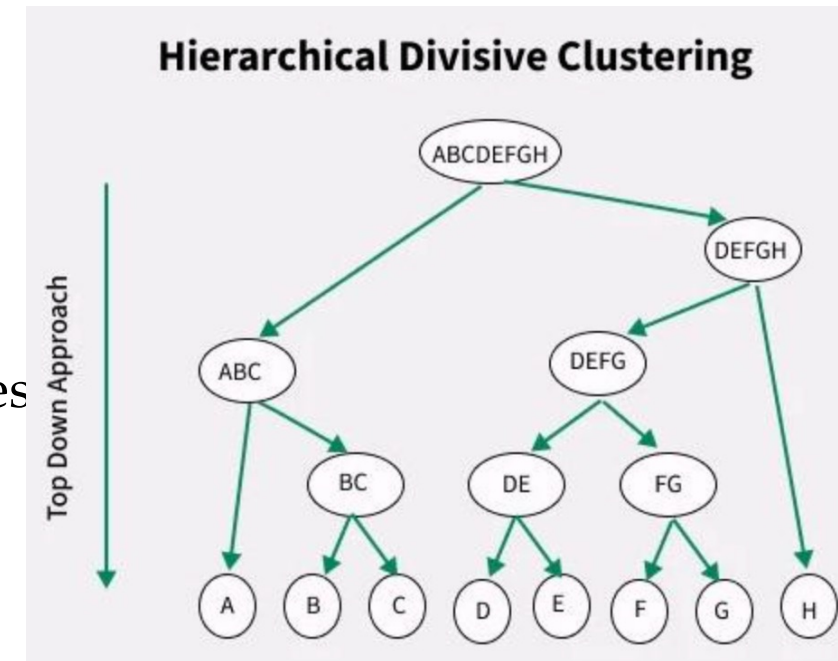
From Distances to Clusters

Divisive hierarchical clustering

Divisive hierarchical clustering is a top-down hierarchical clustering method that works in the opposite way of agglomerative clustering.

Key Steps :

1. **Initialization:** Start with all data points grouped into one large cluster.
2. **Splitting:** Select the cluster to split, usually the one with the largest dissimilarity or highest internal variance.
3. **Division Criterion:** Split the cluster into two subsets (often using methods such as K-Means, spectral clustering, or distance-based separation). [computational cost, $O(2^n)$]
4. **Repeat:** Continue splitting clusters recursively until each data point forms its own cluster or a threshold number of clusters is reached.



Question: Which one is more computationally expensive?

From Distances to Clusters

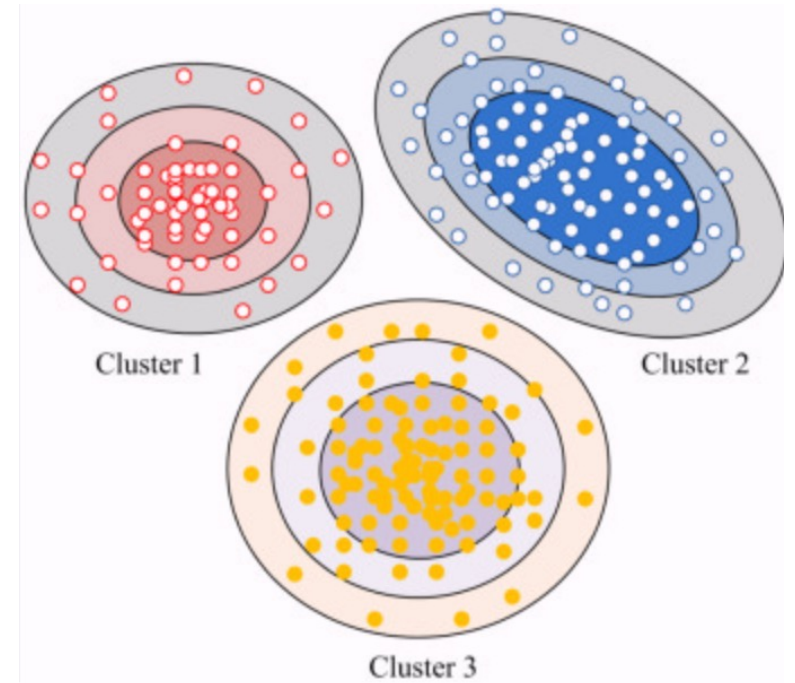
Distribution-based clustering

Distribution-based clustering assumes data is generated from a mixture of probability distributions, such as Gaussian distributions and assigns points to clusters based on statistical likelihood.

This method supports clusters with **flexible shapes and overlaps**, but usually **requires specifying the number of distributions**.

Approach:

- Gaussian Mixture Model (GMM): Fits data as a weighted mixture of Gaussian distributions



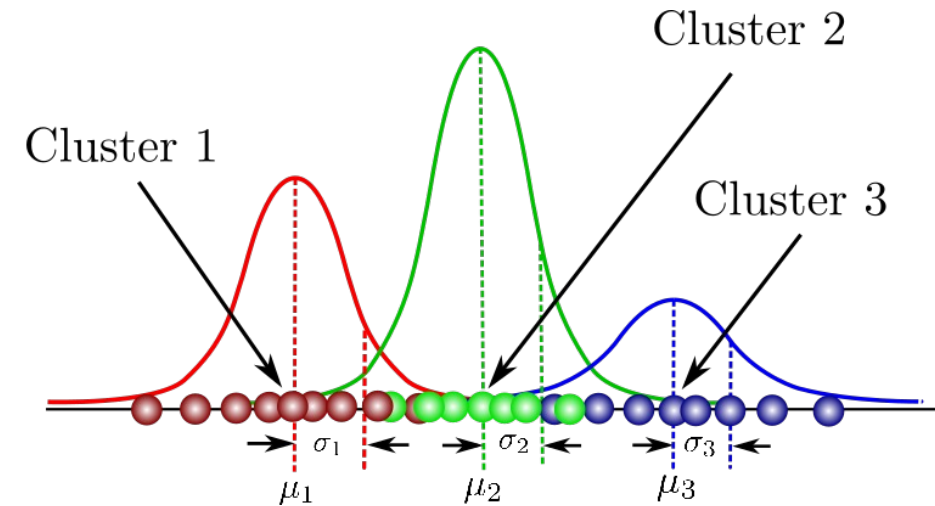
From Distances to Clusters

Gaussian Mixture Model (GMM)

Gaussian Mixture Model (GMM) is a probabilistic clustering algorithm that **assumes the data is generated from a mixture of several Gaussian (normal) distributions**, each representing a cluster.

Key Concepts

- **Gaussian Components:** Each cluster is represented by a Gaussian distribution characterized by its mean (μ) and covariance matrix (Σ).
- **Mixture Weights (π):** Each Gaussian component has an associated weight indicating its relative size or importance in the mixture.
- **Soft Assignment:** Instead of hard labels, each data point has a probability of belonging to each cluster, computed based on the Gaussian densities.

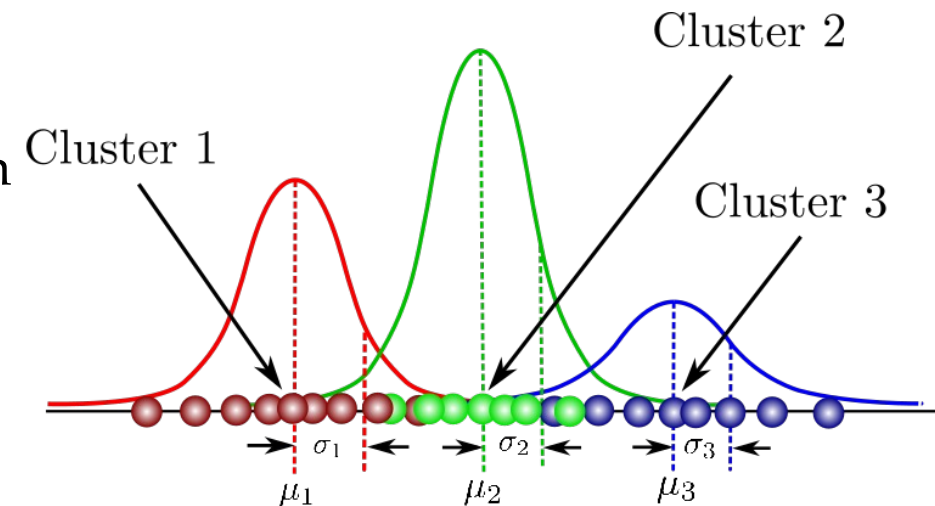


From Distances to Clusters

Gaussian Mixture Model (GMM)

GMM is typically trained using the **Expectation–Maximization (EM) algorithm**, which iteratively refines the parameters:

1. **Initialization:** Choose the number of clusters (k) and initialize the means, covariances, and mixture weights (often using K-Means).
2. **E-Step (Expectation):** Compute the probability that each data point belongs to each Gaussian component.
3. **M-Step (Maximization):** Update the parameters (μ , Σ , π) to maximize the likelihood of the observed data given these probabilities.
4. **Repeat:** Alternate between the E-step and M-step until convergence — i.e., when changes in likelihood become negligible.



Note:

- GMM produces soft cluster memberships, useful when boundaries are ambiguous.
- GMM may perform poorly if the data does **not follow** Gaussian distributions.

From Distances to Clusters

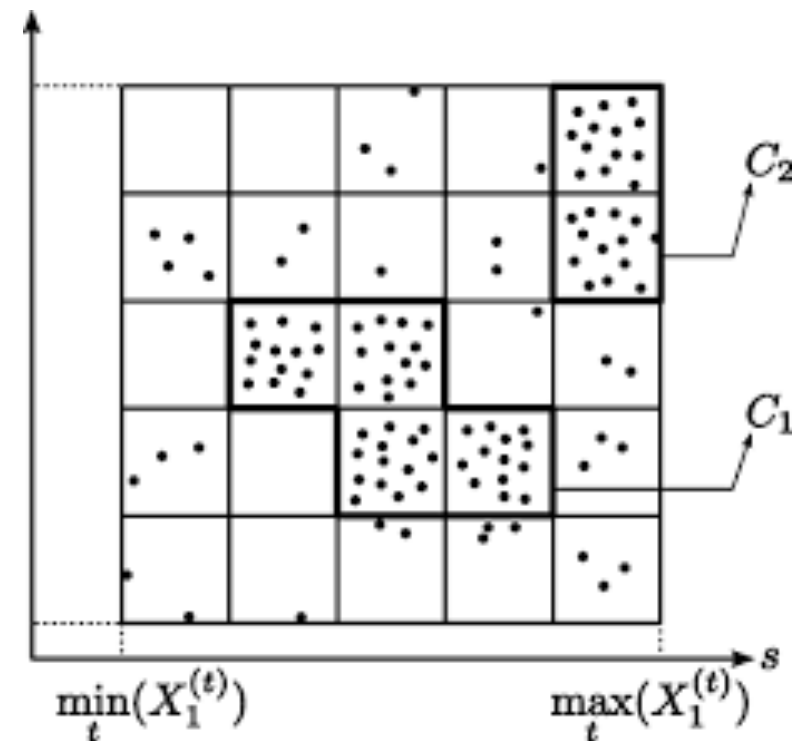
Grid-based clustering

Grid-based clustering is a category of clustering algorithms that divide the data space into a finite number of **uniform cells (grids)** and **perform clustering operations on these cells** instead of directly on the data points.

- This approach is especially **efficient for large datasets** because it reduces the problem's complexity.

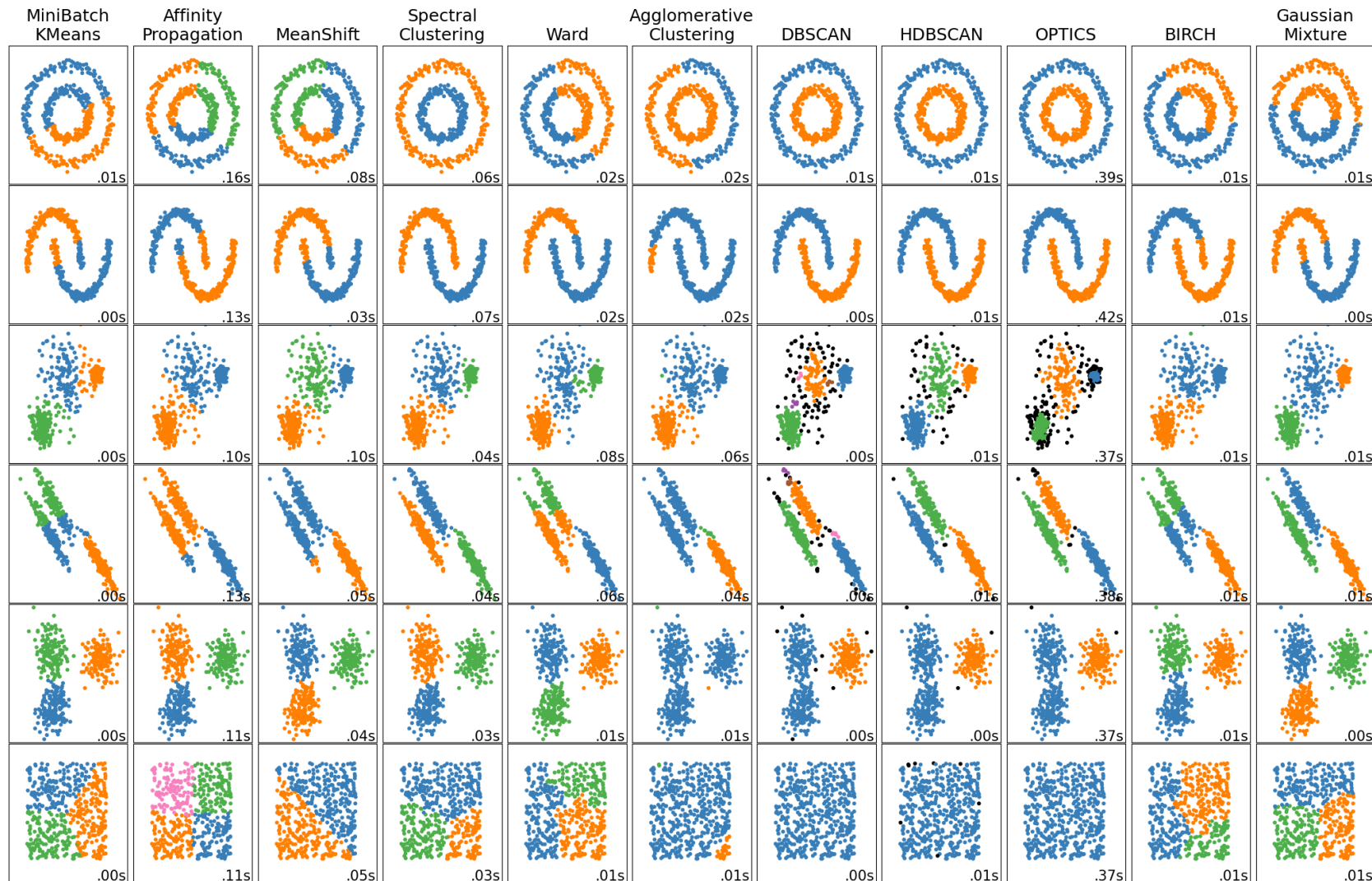
Examples of Grid-Based Methods

- **STING (Statistical Information Grid)**: Uses statistical information stored in a hierarchical grid structure.
- **CLIQUE**: Combines grid-based and density-based methods to discover subspace clusters in high-dimensional data.
- **WaveCluster**: Applies wavelet transformation to identify dense regions.



From Distances to Clusters

Clustering algorithms in scikit-learn



<https://scikit-learn.org/stable/modules/clustering.html>

From Distances to Clusters

Does my clustering perform well?

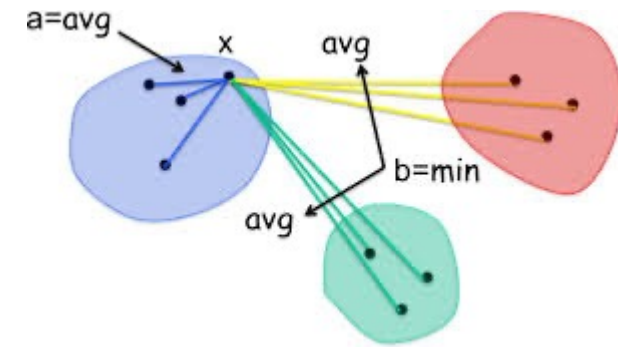
The **silhouette coefficient** is a metric used to evaluate how well samples are clustered. Applicable to **distance-based** clustering algorithms: K-Means, DBSCAN, GMM, Hierarchical Clustering, etc.

How it works

- **Cohesion (a):** This is the average distance of a data point to all other points in its own cluster (**within-cluster similarity**).
- **Separation (b):** This is the average distance of a data point to all points in the nearest neighboring cluster (**between-cluster dissimilarity**).

How to interpret the scores

- 1: The data point is well-clustered and far from other clusters.
- 0: The data point is on or very close to the decision boundary between two clusters.
- -1: The data point is likely misclassified and is closer to another cluster.



The formula: The silhouette coefficient for a single data point i is calculated as:

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

The overall silhouette coefficient is calculated as:

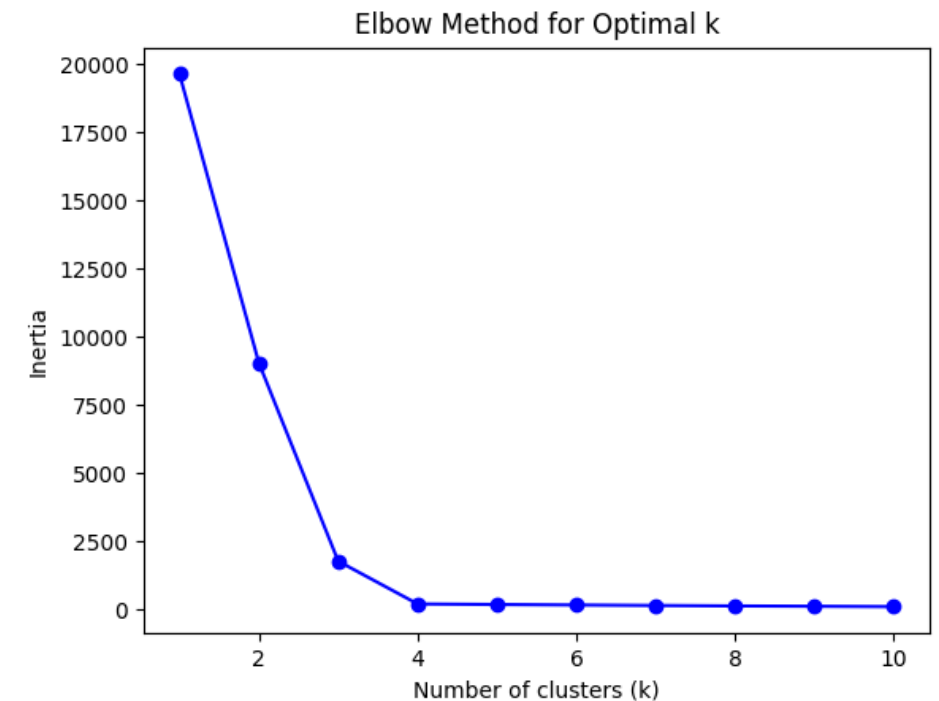
$$s = \frac{1}{n} \sum s_i$$

Python Implementation

Python Implementation

Elbow Method for optimal value of k in K-Means

```
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt
# 1. generate sample data
X, _ = make_blobs(n_samples=300, centers=4,
random_state=42, cluster_std=0.6)
# 2. Calculate inertia of Kmeans using different K
inertias = []
K = range(1, 11) # try k from 1-10
for k in K:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X)
    inertias.append(kmeans.inertia_) # save inertia
results
# 3. Visualizing the Results
plt.plot(K, inertias, 'bo-')
plt.xlabel('Number of clusters (k)')
plt.ylabel('Inertia')
plt.title('Elbow Method for Optimal k')
plt.show()
```



Question: which k should we choose?

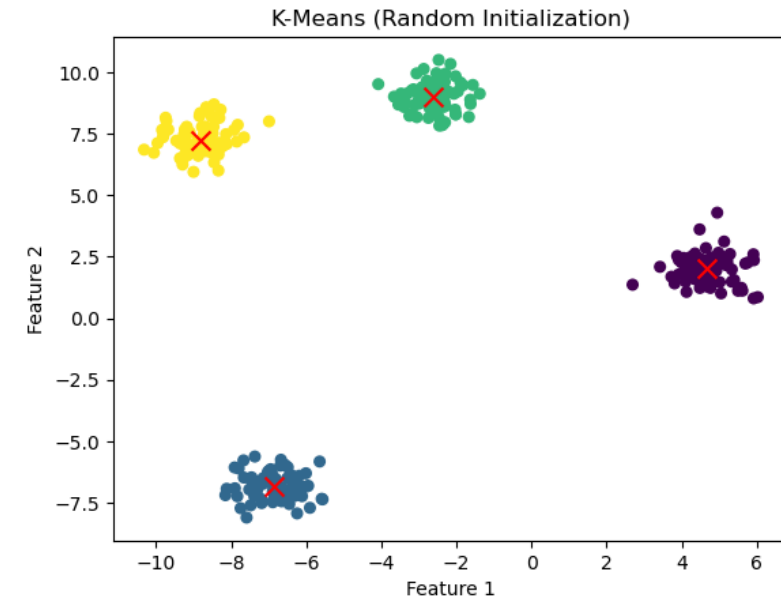
Python Implementation

K-Means

```
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt
# 1. generate sample data
X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.6,
random_state=42)

# 2. ordinary K-Means (by default, sklearn uses kmeans++)
kmeans_random = KMeans(n_clusters=4, init='random', random_state=42)
kmeans_random.fit(X)

# 3. visualization
plt.scatter(X[:, 0], X[:, 1], c=kmeans_random.labels_, s=30,
cmap='viridis')
plt.scatter(kmeans_random.cluster_centers_[:, 0],
kmeans_random.cluster_centers_[:, 1], c='red', s=100, marker='x')
plt.title('K-Means (Random Initialization)')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```



Python Implementation

pip install scikit-learn-extra

More clustering examples

```
from sklearn.cluster import KMeans
from sklearn_extra.cluster import KMedoids
from sklearn.datasets import make_blobs
# generate sample data
X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.6, random_state=42)

kmeans_plus = KMeans(n_clusters=4, init='k-means++', random_state=42) # kmeans ++
kmeans_plus_labels=kmeans_plus.fit(X)

kmedoids = KMedoids(n_clusters=4, random_state=42, method='pam') # kmedoids
Kmedoids_labels=kmedoids.fit(X)

dbscan = DBSCAN(eps=0.5, min_samples=5) # dbscan
labels_dbscan = dbscan.fit_predict(X) ← You can get labels by using model.fit_predict(X)

gmm = GaussianMixture(n_components=3, random_state=42) # gmm
labels_gmm = gmm.fit_predict(X)
```


Python Implementation

Clustering evaluation using the silhouette coefficient

```
from sklearn.metrics import silhouette_score
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

# generate sample data
X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.6, random_state=42)

# Kmeans clustering (kmeans++)
kmeans = KMeans(n_clusters=4, random_state=42)
labels = kmeans.fit_predict(X)

# calculate the silhouette coefficient
score = silhouette_score(X, labels)
print(f"Silhouette Score: {score:.4f}")
```



Just one line of code

Tutorial

